

ALGORITHM DESIGN & IMPLEMENTATION

40 Practical Problems

Java solutions • patterns • brute force → better → best

A screen-reading companion to the practical list in the supplied image. Each chapter states the problem, names its core pattern, compares three approaches, gives Java code, and notes important edge cases.

Prepared September 2026

How to use this workbook

Before reading the code, try the problem and write a small example by hand. Then compare the baseline, improved, and best approach. “Best” means best for the stated constraints; an output-heavy problem may have the same asymptotic bound at every level because every answer must be produced.

Code blocks are Java method/class snippets. Add `import java.util.*;` to the enclosing class. Tree chapters share this node definition:

```
static class TreeNode {  
    int val;  
    TreeNode left, right;  
    TreeNode(int val) { this.val = val; }  
}
```

Assumptions made where the list is brief

- Practical 3 uses M items distributed over N numbered positions, starting at K, with wraparound; it is not Josephus elimination.
- Practical 4 uses Two Sum as the concrete example because “solve a problem using brute force” does not specify a particular prompt.
- Practical 19 uses most-frequent library word/token with lexicographic tie-breaking as a representative hashing exercise. Replace it with the instructor’s exact “Noise in the Library” statement if different.
- Practical 20 means split all given weights into two equal-total groups. Practical 24 assumes a rectangular obstacle grid and includes both backtracking and shortest-path approaches.
- Practical 26 uses the standard square Rat-in-a-Maze version with four-direction movement and no revisiting cells. Practical 33 means running min/max over the entire stream prefix; a fixed window needs monotonic deques.
- Where no distinct asymptotic “better” algorithm exists, the progression shows a clearer baseline, an implementation improvement, and the constraint-appropriate solution.

Practical solutions

The chapters follow the exact 1–40 order from the supplied list.

01. Prime factorization

Problem: Given a positive integer n , return its prime factors with multiplicity. For example, $84 = 2 \times 2 \times 3 \times 7$.

Pattern: Factor stripping; divisors only need to be tested through $\sqrt{\text{remaining } n}$.

1. Brute force: try every possible divisor

$O(n)$ time, $O(\log n)$ output space

Test every integer from 2 upward and divide it out whenever it is a factor.

```
static List<Long> factorsBrute(long n) {
    List<Long> out = new ArrayList<>();
    for (long d = 2; d <= n; d++) {
        if (n % d == 0) {
            out.add(d);
            while (n % d == 0) n /= d;
        }
    }
    return out;
}
```

2. Better: divide by 2, then odd divisors

$O(\sqrt{n})$ time, $O(\log n)$ output space

After 2, only odd candidates are needed. Once $d \cdot d$ exceeds the remaining n , that remainder is prime.

```
static List<Long> factorsSqrt(long n) {
    List<Long> out = new ArrayList<>();
    for (long d = 2; d <= n / d; d = (d == 2 ? 3 : d + 2)) {
        while (n % d == 0) { out.add(d); n /= d; }
    }
    if (n > 1) out.add(n);
    return out;
}
```

3. Best for many bounded queries: smallest-prime-factor table

Precompute $O(M \log \log M)$; each query $O(\log n)$

Build SPF once up to the largest allowed value M , then repeatedly divide by $\text{spf}[n]$. For one very large n , the sqrt method is simpler.

```
static int[] buildSpf(int max) {
    int[] spf = new int[max + 1];
    for (int i = 2; i <= max; i++) if (spf[i] == 0) {
        spf[i] = i;
        if ((long)i * i <= max)
            for (int j = i * i; j <= max; j += i)
                if (spf[j] == 0) spf[j] = i;
    }
    return spf;
}

static List<Integer> factorsWithSpf(int n, int[] spf) {
    List<Integer> out = new ArrayList<>();
    while (n > 1) { out.add(spf[n]); n /= spf[n]; }
    return out;
}
```

Tips and edge cases

Use $d \leq n / d$ instead of $d \cdot d \leq n$ to avoid multiplication overflow. Decide whether the output should repeat a prime ($84 \rightarrow 2, 2, 3, 7$) or list each prime once. The SPF method is for many queries in a known range.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Distinct Prime Factors Of Product Of Array](#)

02. GCD of two numbers

Problem: Given nonnegative integers a and b , return their greatest common divisor. Use $\text{gcd}(0, x) = x$.

Pattern: Divisibility; Euclidean remainder invariant $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$.

1. Brute force: test downward from $\min(a, b)$

$O(\min(a, b))$ time, $O(1)$ space

The first number that divides both is the answer.

```
static long gcdBrute(long a, long b) {
    long limit = Math.min(a, b);
    for (long d = limit; d >= 1; d--)
        if (a % d == 0 && b % d == 0) return d;
    return 0;
}
```

2. Better: repeated subtraction

$O(\max(a, b))$ worst-case time, $O(1)$ space

Replace the larger value by the difference until the values meet. It is correct but slow for uneven magnitudes.

```
static long gcdSubtract(long a, long b) {
    if (a == 0) return b;
    if (b == 0) return a;
    while (a != b) {
        if (a > b) a -= b;
        else b -= a;
    }
    return a;
}
```

3. Best: Euclidean algorithm

$O(\log \min(a, b))$ time, $O(1)$ space

Remainders shrink quickly. This is the standard implementation.

```
static long gcd(long a, long b) {
    a = Math.abs(a); b = Math.abs(b);
    while (b != 0) {
        long r = a % b;
        a = b;
        b = r;
    }
    return a;
}
```

Tips and edge cases

Normalize negative inputs if they are allowed. To compute lcm safely, use $(a / \text{gcd}(a, b)) * b$ rather than $a * b / \text{gcd}$.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Find Greatest Common Divisor Of Array](#)

03. Distribute M items in a circle of N positions

Problem: There are N positions numbered 1 to N. Distribute M items to consecutive positions, starting at position K and wrapping from N back to 1. Return the position that receives the last item. Example: N=5, M=8, K=2 gives position 4.

Pattern: Circular indexing; convert to zero-based, add M - 1 steps, take modulo N, then convert back to one-based.

1. Brute force: advance one item at a time

O(M) time, O(1) space

Move one position per item and wrap around when reaching N.

```
static int distributeBrute(int n, int m, int k) {
    int pos = k;
    for (int item = 1; item < m; item++) {
        pos++;
        if (pos > n) pos = 1;
    }
    return pos;
}
```

2. Better: simulate with a wrap condition

O(M) time, O(1) space

Use zero-based indexing and keep the position modulo n after each step.

```
static int distributeLoop(int n, int m, int k) {
    if (n <= 0 || m <= 0 || k < 1 || k > n) throw new IllegalArgumentException();
    int pos = k - 1;
    for (int i = 1; i < m; i++) pos = (pos + 1) % n;
    return pos + 1;
}
```

3. Best: direct modular formula

O(1) time, O(1) space

The first item is at K, so the last is K + M - 1 steps from the start, wrapped around the circle.

```
static int distribute(int n, int m, int k) {
    if (n <= 0 || m <= 0 || k < 1 || k > n) throw new IllegalArgumentException();
    return (int)((long)k - 1 + (long)m - 1) % n + 1;
}
```

Tips and edge cases

Do not confuse this with Josephus elimination: here items go to consecutive positions, and nobody is removed. Long arithmetic avoids overflow before modulo. If the prompt uses a different starting-index convention, adjust the formula.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Find The Winner Of The Circular Game](#) (related cyclic elimination; different task)

04. Brute force method (worked example: Two Sum)

Problem: Given an integer array and target K, return indices of two different elements whose values sum to K, or an empty array if no pair exists.

Pattern: Brute force means enumerate the valid candidate space directly, then check each candidate. The same pattern appears in pairs, subarrays, and subsets.

1. Brute force: inspect every pair

$O(n^2)$ time, $O(1)$ extra space

Try every $i < j$ and stop when the pair reaches K.

```
static int[] twoSumBrute(int[] a, int k) {
    for (int i = 0; i < a.length; i++)
        for (int j = i + 1; j < a.length; j++)
            if ((long)a[i] + a[j] == k) return new int[]{i, j};
    return new int[0];
}
```

2. Better: sort value-index pairs and use two pointers

$O(n \log n)$ time, $O(n)$ space

Sorting makes the sum monotone as the two pointers move. Keep original indices alongside values.

```
static int[] twoSumSortedCopy(int[] a, int k) {
    int[][] x = new int[a.length][2];
    for (int i = 0; i < a.length; i++) { x[i][0] = a[i]; x[i][1] = i; }
    Arrays.sort(x, Comparator.comparingInt(v -> v[0]));
    int l = 0, r = x.length - 1;
    while (l < r) {
        long sum = (long)x[l][0] + x[r][0];
        if (sum == k) return new int[]{x[l][1], x[r][1]};
        if (sum < k) l++; else r--;
    }
    return new int[0];
}
```

3. Best for one unsorted query: hash complements

$O(n)$ expected time, $O(n)$ space

At each value, look for K-value among earlier values, then store the current index.

```
static int[] twoSumHash(int[] a, int k) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        long need = (long)k - a[i];
        if (need >= Integer.MIN_VALUE && need <= Integer.MAX_VALUE &&
            seen.containsKey((int)need))
            return new int[]{seen.get((int)need), i};
        seen.putIfAbsent(a[i], i);
    }
    return new int[0];
}
```

Tips and edge cases

This practical names a technique, not a unique problem; Two Sum is the illustrative example. Define whether the result needs values or indices. Use long for pair-sum arithmetic.

LeetCode matches or related practice: [Two Sum](#)

05. Example of a randomized algorithm: randomized quicksort

Problem: Sort an integer array. Demonstrate how choosing a pivot randomly avoids input-order-dependent bad splits in quicksort; the result is still deterministic even though the pivot choices are random.

Pattern: Randomized divide and conquer; partition around a random pivot, then recursively sort the two sides.

1. Brute force: insertion sort

$O(n^2)$ time, $O(1)$ space

A simple comparison sort provides a baseline and works well only on small or nearly sorted arrays.

```
static void insertionSort(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int x = a[i], j = i - 1;
        while (j >= 0 && a[j] > x) { a[j + 1] = a[j]; j--; }
        a[j + 1] = x;
    }
}
```

2. Better: quicksort with a fixed last-element pivot

Average $O(n \log n)$, worst $O(n^2)$

Partition in place and recurse. An already sorted input can repeatedly produce a highly unbalanced split.

```
static void quickSortFixed(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int p = partitionLast(a, lo, hi);
    quickSortFixed(a, lo, p - 1); quickSortFixed(a, p + 1, hi);
}
static int partitionLast(int[] a, int lo, int hi) {
    int pivot = a[hi], p = lo;
    for (int i = lo; i < hi; i++) if (a[i] <= pivot) swap(a, p++, i);
    swap(a, p, hi); return p;
}
static void swap(int[] a, int i, int j) { int t=a[i]; a[i]=a[j]; a[j]=t; }
```

3. Best average behavior: randomized 3-way quicksort

Expected $O(n \log n)$, worst $O(n^2)$; $O(\log n)$ expected stack

Choose a random pivot and group values into less/equal/greater regions. The 3-way partition handles duplicate-heavy inputs efficiently.

```
static final Random RNG = new Random();
static void quickSortRandom(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int pivot = a[lo + RNG.nextInt(hi - lo + 1)];
    int lt = lo, i = lo, gt = hi;
    while (i <= gt) {
        if (a[i] < pivot) swap(a, lt++, i++);
        else if (a[i] > pivot) swap(a, i, gt--);
        else i++;
    }
    quickSortRandom(a, lo, lt - 1); quickSortRandom(a, gt + 1, hi);
}
```

Tips and edge cases

Randomization improves expected balance; it does not remove the theoretical $O(n^2)$ worst case. Seed Random when reproducible traces are useful. For library code, Arrays.sort is usually preferable.

LeetCode matches or related practice: [Sort An Array](#)

06. Divide-and-conquer sort an array

Problem: Sort an integer array in ascending order using a divide-and-conquer algorithm.

Pattern: Divide the array into smaller ranges, solve each range recursively, then combine the sorted results.

1. Brute force: selection sort

$O(n^2)$ time, $O(1)$ space

Repeatedly select the smallest remaining value. It is easy to trace but does not exploit divide and conquer.

```
static void selectionSort(int[] a) {
    for (int i = 0; i < a.length; i++) {
        int m = i;
        for (int j = i + 1; j < a.length; j++) if (a[j] < a[m]) m = j;
        int t=a[i]; a[i]=a[m]; a[m]=t;
    }
}
```

2. Better: merge sort

$O(n \log n)$ time, $O(n)$ space

Sort both halves recursively and merge them stably into a reusable buffer.

```
static void mergeSort(int[] a) { mergeSort(a, new int[a.length], 0, a.length); }
static void mergeSort(int[] a, int[] tmp, int lo, int hi) {
    if (hi - lo <= 1) return;
    int mid = (lo + hi) >> 1;
    mergeSort(a, tmp, lo, mid); mergeSort(a, tmp, mid, hi);
    int i=lo, j=mid, p=lo;
    while (i<mid || j<hi) tmp[p++] = (j==hi || (i<mid && a[i]<=a[j])) ? a[i++] : a[j++];
    System.arraycopy(tmp, lo, a, lo, hi-lo);
}
```

3. Best for average in-place sorting: randomized 3-way quicksort

Expected $O(n \log n)$, $O(\log n)$ expected stack

Partition by a random pivot; recurse only on values smaller and larger than the pivot.

```
static void quickSort(int[] a) { quick(a, 0, a.length - 1, new Random()); }
static void quick(int[] a, int lo, int hi, Random rnd) {
    while (lo < hi) {
        int pivot = a[lo + rnd.nextInt(hi-lo+1)], lt=lo, i=lo, gt=hi;
        while (i<=gt) {
            if (a[i]<pivot) { int t=a[lt];a[lt++]=a[i];a[i++]=t; }
            else if (a[i]>pivot) { int t=a[i];a[i]=a[gt];a[gt--]=t; }
            else i++;
        }
        if (lt-lo < hi-gt) { quick(a,lo,lt-1,rnd); lo=gt+1; }
        else { quick(a,gt+1,hi,rnd); hi=lt-1; }
    }
}
```

Tips and edge cases

Merge sort guarantees $O(n \log n)$ and stability. Quicksort is usually faster in memory but can degrade in its worst case. If stability or strict worst-case bounds are required, choose accordingly.

LeetCode matches or related practice: [Sort An Array](#)

07. Find a pair with sum K using two pointers

Problem: Given a sorted integer array and target K, return whether two distinct values sum to K. The array is sorted in nondecreasing order.

Pattern: Two pointers on sorted input; increasing the left value raises the sum, while decreasing the right value lowers it.

1. Brute force: test every pair

$O(n^2)$ time, $O(1)$ space

Try every $i < j$; sorting is unused in this baseline.

```
static boolean pairBrute(int[] a, int k) {
    for(int i=0; i<a.length; i++){
        for(int j=i+1; j<a.length; j++){
            if((long)a[i]+a[j]==k) return true;
        }
    }
    return false;
}
```

2. Better: binary search for each complement

$O(n \log n)$ time, $O(1)$ extra space

For each index i , binary-search the suffix for $K-a[i]$.

```
static boolean pairBinary(int[] a, int k) {
    for(int i=0; i<a.length; i++){
        long need=(long)k-a[i];
        if(need<Integer.MIN_VALUE||need>Integer.MAX_VALUE) continue;
        int j=Arrays.binarySearch(a, i+1, a.length, (int)need);
        if(j>=0) return true;
    }
    return false;
}
```

3. Best: two-pointer scan

$O(n)$ time, $O(1)$ space

Start at both ends and move the pointer that can bring the sum toward K.

```
static boolean pairTwoPointers(int[] a, int k) {
    int l=0, r=a.length-1;
    while(l<r){
        long sum=(long)a[l]+a[r];
        if(sum==k) return true;
        if(sum<k) l++; else r--;
    }
    return false;
}
```

Tips and edge cases

Two pointers are valid because the input is sorted. If the array is unsorted, sort a copy (while retaining indices if needed) or use a HashSet. Use long for the sum.

LeetCode matches or related practice: [Two Sum If Input Array Is Sorted](#)

08. Maximum sum subarray of fixed size k

Problem: Given an integer array and $1 \leq k \leq n$, return the largest sum among all contiguous subarrays of exactly k elements.

Pattern: Fixed-size sliding window; subtract the element leaving the window and add the element entering it.

1. Brute force: sum each window independently

$O(nk)$ time, $O(1)$ space

For each start, add its k values and compare with the best sum.

```
static long maxWindowBrute(int[] a, int k) {
    if(k <= 0 || k > a.length) throw new IllegalArgumentException();
    long best = Long.MIN_VALUE;
    for(int s = 0; s + k <= a.length; s++) { long sum = 0; for(int i = s; i < s + k; i++) sum += a[i];
        best = Math.max(best, sum); }
    return best;
}
```

2. Better: prefix sums

$O(n)$ time, $O(n)$ space

A window sum is $\text{prefix}[\text{right}+1] - \text{prefix}[\text{left}]$.

```
static long maxWindowPrefix(int[] a, int k) {
    long[] pref = new long[a.length + 1];
    for(int i = 0; i < a.length; i++) pref[i + 1] = pref[i] + a[i];
    long best = Long.MIN_VALUE;
    for(int s = 0; s + k <= a.length; s++) best = Math.max(best, pref[s + k] - pref[s]);
    return best;
}
```

3. Best: rolling sum

$O(n)$ time, $O(1)$ extra space

Compute the first window once; then update the sum by removing its left value and adding the new right value.

```
static long maxWindow(int[] a, int k) {
    if(k <= 0 || k > a.length) throw new IllegalArgumentException();
    long sum = 0; for(int i = 0; i < k; i++) sum += a[i];
    long best = sum;
    for(int r = k; r < a.length; r++) { sum += (long)a[r] - a[r - k]; best = Math.max(best, sum); }
    return best;
}
```

Tips and edge cases

Use long because k array values can overflow int. Initialize best to the first window sum (not zero), since all numbers may be negative.

LeetCode matches or related practice: [Maximum Average Subarray I](#)

09. Longest Common Subsequence (LCS)

Problem: Given strings a and b, return the length of their longest subsequence that appears in both strings in the same relative order. Characters need not be contiguous.

Pattern: Two-dimensional dynamic programming. If characters match, take the diagonal plus one; otherwise take the better of skipping one character from either string.

1. Brute force: recursive choices

Exponential time, $O(m \cdot n)$ stack

For each pair of positions, branch by skipping a character; when equal, either match them or explore skips.

```
static int lcsBrute(String a,String b,int i,int j) {
    if(i==a.length()||j==b.length()) return 0;
    if(a.charAt(i)==b.charAt(j)) return 1+lcsBrute(a,b,i+1,j+1);
    return Math.max(lcsBrute(a,b,i+1,j),lcsBrute(a,b,i,j+1));
}
```

2. Better: memoized recursion

$O(mn)$ time, $O(mn)$ space

Cache each state (i,j); each pair is computed once.

```
static int lcsMemo(String a,String b) {
    int[][] memo=new int[a.length()][b.length()];
    for(int[] row:memo) Arrays.fill(row,-1);
    return lcsMemo(a,b,0,0,memo);
}
static int lcsMemo(String a,String b,int i,int j,int[][] m) {
    if(i==a.length()||j==b.length()) return 0;
    if(m[i][j]!=-1) return m[i][j];
    return
        m[i][j]=a.charAt(i)==b.charAt(j)?1+lcsMemo(a,b,i+1,j+1,m):Math.max(lcsMemo(a,b,i+1,j,m),lcsMemo(a,b,i,j+1,m));
}
```

3. Best for length only: rolling-row DP

$O(mn)$ time, $O(\min(m,n))$ space

Fill rows bottom-up and keep only the current and next rows. Swap inputs so the row width is the shorter string.

```
static int lcs(String a,String b) {
    if(b.length()>a.length()){ String t=a;a=b;b=t; }
    int[] next=new int[b.length()+1], cur=new int[b.length()+1];
    for(int i=a.length()-1;i>=0;i--){
        for(int j=b.length()-1;j>=0;j--){
            cur[j]=a.charAt(i)==b.charAt(j)?1+next[j+1]:Math.max(next[j],cur[j+1]);
        }
        int[] t=next;next=cur;cur=t;Arrays.fill(cur,0);
    }
    return next[0];
}
```

Tips and edge cases

The problem asks for length; reconstructing the actual subsequence needs parent/choice storage. Distinguish subsequence from substring: a subsequence can skip characters.

LeetCode matches or related practice: [Longest Common Subsequence](#)

10. Rotate an array right by d positions

Problem: Move every array value d places to the right, wrapping values from the end to the front. Example: [1,2,3,4,5], d=2 becomes [4,5,1,2,3].

Pattern: Array rotation; normalize d modulo n; reversal turns a shift into three in-place reversals.

1. Brute force: rotate one place repeatedly

O(nd) time, O(1) extra space

Repeat a one-step right shift d times.

```
static void rotateBrute(int[] a, int d) {
    int n = a.length;
    if (n == 0) return;
    d = Math.floorMod(d, n);
    for (int step = 0; step < d; step++) {
        int last = a[n - 1];
        for (int i = n - 1; i > 0; i--) a[i] = a[i - 1];
        a[0] = last;
    }
}
```

2. Better: write into a second array

O(n) time, O(n) extra space

Place each original index i at (i+d) mod n, then copy back.

```
static void rotateBuffer(int[] a, int d) {
    int n = a.length;
    if (n == 0) return;
    d = Math.floorMod(d, n);
    int[] b = new int[n];
    for (int i = 0; i < n; i++) b[(i + d) % n] = a[i];
    System.arraycopy(b, 0, a, 0, n);
}
```

3. Best: reverse three sections

O(n) time, O(1) extra space

Reverse all, reverse the first d values, then reverse the remaining suffix.

```
static void rotateReverse(int[] a, int d) {
    int n = a.length;
    if (n == 0) return;
    d = Math.floorMod(d, n);
    reverse(a, 0, n - 1);
    reverse(a, 0, d - 1);
    reverse(a, d, n - 1);
}

static void reverse(int[] a, int l, int r) {
    while (l < r) { int t = a[l]; a[l++] = a[r]; a[r--] = t; }
}
```

Tips and edge cases

For a left rotation by d, use right distance n - d after normalization. Test d=0, d=n, d>n, and an empty array. This rotates the array; shell rotation of matrix rings is a separate problem below.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Rotate Array](#)

11. Check whether two strings are anagrams

Problem: Return true if strings *s* and *t* contain exactly the same characters with the same multiplicities. These examples assume lowercase English letters; adapt the alphabet or use a map for general Unicode.

Pattern: Frequency counting; anagrams have equal character histograms.

1. Brute force: sort both strings

$O(n \log n)$ time, $O(n)$ space

If sorted character arrays match, the strings are anagrams.

```
static boolean anagramSort(String s,String t) {
    if(s.length()!=t.length()) return false;
    char[] a=s.toCharArray(), b=t.toCharArray(); Arrays.sort(a);Arrays.sort(b);
    return Arrays.equals(a,b);
}
```

2. Better: frequency map

$O(n)$ expected time, $O(u)$ space

Increment counts for *s* and decrement for *t*; all counts must return to zero.

```
static boolean anagramMap(String s,String t) {
    if(s.length()!=t.length()) return false;
    Map<Character,Integer> f=new HashMap<>();
    for(char c:s.toCharArray()) f.merge(c,1,Integer::sum);
    for(char c:t.toCharArray()) { int v=f.getOrDefault(c,0)-1; if(v<0)return false; f.put(c,v);
    }
    return true;
}
```

3. Best for lowercase English: fixed frequency array

$O(n)$ time, $O(1)$ space

Use 26 counters and early reject when a count becomes negative.

```
static boolean isAnagram(String s,String t) {
    if(s.length()!=t.length()) return false;
    int[] f=new int[26];
    for(int i=0;i<s.length();i++){ f[s.charAt(i)-'a']++; f[t.charAt(i)-'a']--; }
    for(int x:f) if(x!=0)return false;
    return true;
}
```

Tips and edge cases

Clarify case sensitivity, spaces, punctuation, and character set. For arbitrary Java char values use a 65,536-entry array; for Unicode code points, count code points in a map.

LeetCode matches or related practice: [Valid Anagram](#)

12. Greedy interval scheduling (maximum number of non-overlapping intervals)

Problem: Given intervals [start,end), select the largest possible number that do not overlap. This version treats an interval ending at t as compatible with one starting at t.

Pattern: Greedy choice by earliest finishing time; selecting the interval that frees the resource earliest leaves maximum room for later choices.

1. Brute force: enumerate subsets

$O(2^n * n \log n)$ time

Try each subset, sort its selected intervals by start, check compatibility, and retain the largest valid subset.

```
static int scheduleBrute(int[][] a) {
    int best=0,n=a.length;
    for(int mask=0;mask<(1<<n);mask++){
        List<int[]> pick=new ArrayList<>();for(int
            i=0;i<n;i++){if((mask&(1<<i))!=0)pick.add(a[i]);
        pick.sort(Comparator.comparingInt(x->x[0]));boolean ok=true;
        for(int i=1;i<pick.size();i++){if(pick.get(i)[0]<pick.get(i-1)[1]){ok=false;break;}
        if(ok)best=Math.max(best,pick.size());
    }
    return best;
}
```

2. Better: dynamic programming after sorting by end

$O(n^2)$ time, $O(n)$ space

For each interval, find the best compatible earlier interval and extend its answer.

```
static int scheduleDp(int[][] input) {
    int[][] a=Arrays.stream(input).map(int[]::clone).toArray(int[][]::new);
    Arrays.sort(a,Comparator.comparingInt(x->x[1])); int n=a.length;
    int[] dp=new int[n];
    for(int i=0;i<n;i++){
        dp[i]=1;
        for(int j=0;j<i;j++) if(a[j][1]<=a[i][0]) dp[i]=Math.max(dp[i],dp[j]+1);
    }
    int best=0;for(int x:dp)best=Math.max(best,x);return best;
}
```

3. Best: sort by end, take each compatible interval

$O(n \log n)$ time, $O(n)$ copy

The earliest-finish greedy choice is optimal for unweighted interval scheduling.

```
static List<int[]> scheduleGreedy(int[][] input) {
    int[][] a=Arrays.stream(input).map(int[]::clone).toArray(int[][]::new);
    Arrays.sort(a,Comparator.comparingInt(x->x[1]));
    List<int[]> chosen=new ArrayList<>(); int end=Integer.MIN_VALUE;
    for(int[] x:a) if(x[0]>=end){chosen.add(x);end=x[1];}
    return chosen;
}
```

Tips and edge cases

Use the right tie rule for the statement: closed intervals often require start > previous end, while half-open intervals allow equality. Weighted intervals need a different DP.

LeetCode matches or related practice: [Non Overlapping Intervals](#) • [Minimum Number Of Arrows To Burst Balloons](#)

13. Fractional knapsack

Problem: Given items with positive weights and values and a capacity W , maximize total value when any fraction of an item may be taken.

Pattern: Greedy by value-to-weight ratio; take the highest-value density first, using a fraction of the final item if needed.

1. Brute force: try each item ordering

$O(n! \cdot n)$ time

For a small n , enumerate orders, greedily fill in that order, and keep the best value. This is exact but impractical.

```
static double fractionalBrute(int[] w,int[] v,double cap) {
    boolean[] used=new boolean[w.length];
    return permuteFill(w,v,cap,used,0,0.0);
}
static double permuteFill(int[] w,int[] v,double cap,boolean[] used,int depth,double value) {
    if(depth==w.length||cap==0)return value;
    double best=value;
    for(int i=0;i<w.length;i++){if(!used[i]){
        used[i]=true; double take=Math.min(1.0,cap/w[i]);
        best=Math.max(best,permuteFill(w,v,cap-take*w[i],used,depth+1,value+take*v[i]));
        used[i]=false;
    }
    }
    return best;
}
```

2. Better: selection-sort items by density

$O(n^2)$ time, $O(n)$ auxiliary space

Repeatedly choose the remaining item with greatest value/weight ratio and take it.

```
static double fractionalSelect(int[] w,int[] v,double cap) {
    boolean[] used=new boolean[w.length]; double ans=0;
    for(int z=0;z<w.length&&cap>0;z++){
        int best=-1;
        for(int i=0;i<w.length;i++){
            if(!used[i]&&(best<0||((double)v[i]/w[i]>(double)v[best]/w[best]))best=i;
            used[best]=true; double f=Math.min(1.0,cap/w[best]); ans+=f*v[best]; cap-=f*w[best];
        }
    }
    return ans;
}
```

3. Best: sort ratios once

$O(n \log n)$ time, $O(n)$ item references

Sort by descending value/weight, then take whole items until the last partial item.

```
static double fractionalKnapsack(int[] w,int[] v,double cap) {
    Integer[] id=new Integer[w.length];for(int i=0;i<id.length;i++)id[i]=i;
    Arrays.sort(id,(i,j)->Double.compare((double)v[j]/w[j],(double)v[i]/w[i]));
    double ans=0;
    for(int i:id){if(cap<=0)break;double f=Math.min(1.0,cap/w[i]);ans+=f*v[i];cap-=f*w[i];}
    return ans;
}
```

Tips and edge cases

This greedy proof applies to fractional knapsack because a high-density item can replace an equal weight of a lower-density item. It does not solve 0/1 knapsack, where items cannot be split.

LeetCode matches or related practice: [Maximum Units On A Truck](#)

14. Schedule jobs with deadlines to maximize profit

Problem: Each job takes one unit of time and has a deadline and profit. Schedule at most one job per time slot, completing a job no later than its deadline, to maximize total profit.

Pattern: Greedy by descending profit; place each job in the latest free slot at or before its deadline. A disjoint-set structure skips full slots.

1. Brute force: enumerate job subsets and slots

Exponential time

For each small instance, try schedules and keep the maximum-profit feasible assignment.

```
static int jobBrute(int[][] jobs,int i,boolean[] used,int time) {
    if(i==jobs.length)return 0;
    int best=jobBrute(jobs,i+1,used,time);
    for(int s=1;s<=jobs[i][0]&& s<used.length;s++){if(!used[s]){
        used[s]=true;best=Math.max(best,jobs[i][1]+jobBrute(jobs,i+1,used,time));used[s]=false;
    }
    return best;
}
```

2. Better: sort by profit and scan slots backward

$O(nD)$ time after sorting, $O(D)$ space

For each high-profit job, search backward from its deadline for the first empty slot.

```
static int jobScheduleScan(int[][] input) {
    int[][] a=Arrays.stream(input).map(int[]::clone).toArray(int[][]::new);
    Arrays.sort(a,(x,y)->Integer.compare(y[1],x[1]));
    int d=0;for(int[] j:a)d=Math.max(d,j[0]); boolean[] slot=new boolean[d+1]; int profit=0;
    for(int[] j:a)for(int
        s=Math.min(d,j[0]);s>0;s--){if(!slot[s]){slot[s]=true;profit+=j[1];break;}}
    return profit;
}
```

3. Best: profit order plus disjoint-set slot search

$O(n \log n + n \alpha(D))$ time, $O(D)$ space

Find the latest available slot $\leq$ deadline and union it with the previous slot after occupying it.

```
static int jobSchedule(int[][] input) {
    int[][] a=Arrays.stream(input).map(int[]::clone).toArray(int[][]::new);
    Arrays.sort(a,(x,y)->Integer.compare(y[1],x[1]));
    int d=0;for(int[] j:a)d=Math.max(d,j[0]);
    int[] parent=new int[d+1];for(int i=0;i<=d;i++)parent[i]=i;
    int profit=0;
    for(int[] j:a){int
        s=find(parent,Math.min(d,j[0]));if(s>0){profit+=j[1];parent[s]=find(parent,s-1);}}
    return profit;
}
static int find(int[] p,int x){return p[x]==x?x:(p[x]=find(p,p[x]));}
```

Tips and edge cases

If jobs have different durations, this simple slot model no longer applies. Clamp deadlines to the number of jobs. For the usual output, track job IDs in the chosen slots as well as total profit.

LeetCode matches or related practice: [Maximum Profit In Job Scheduling](#)

15. Activity selection

Problem: Given activities with start and finish times, choose the maximum number that can be completed by one person, one at a time.

Pattern: Same earliest-finish greedy pattern as unweighted interval scheduling; sort by finish time and accept compatible activities.

1. Brute force: try subsets

$O(2^n * n \log n)$

Enumerate every subset, sort it by start time, and retain the largest non-overlapping subset.

```
static int activityBrute(int[][] a) {
    int best=0,n=a.length;
    for(int mask=0;mask<(1<<n);mask++){
        List<int> pick=new ArrayList<>();for(int
            i=0;i<n;i++){if((mask&(1<<i))!=0)pick.add(a[i]);
        pick.sort(Comparator.comparingInt(x->x[0]));boolean ok=true;
        for(int i=1;i<pick.size();i++){if(pick.get(i)[0]<pick.get(i-1)[1]){ok=false;break;}
        if(ok)best=Math.max(best,pick.size());
    }
    return best;
}
```

2. Better: dynamic programming

$O(n^2)$ time, $O(n)$ space

Sort by finish and compute the best compatible prefix ending at each activity.

```
static int activityDp(int[][] input) {
    int[][] a=Arrays.stream(input).map(int[]::clone).toArray(int[][]::new);
    Arrays.sort(a,Comparator.comparingInt(x->x[1]));int[] dp=new int[a.length];int best=0;
    for(int i=0;i<a.length;i++){dp[i]=1;for(int
        j=0;j<i;j++){if(a[j][1]<=a[i][0])dp[i]=Math.max(dp[i],dp[j]+1);best=Math.max(best,dp[i]);}
    return best;
}
```

3. Best: earliest finish first

$O(n \log n)$ time, $O(n)$ copy

Taking the compatible activity with earliest finish is optimal for unweighted activities.

```
static List<int> activitySelection(int[][] input) {
    int[][] a=Arrays.stream(input).map(int[]::clone).toArray(int[][]::new);
    Arrays.sort(a,Comparator.comparingInt(x->x[1]));List<int> out=new ArrayList<>();int
        end=Integer.MIN_VALUE;
    for(int[] x:a){if(x[0]>=end){out.add(x);end=x[1];}
    return out;
}
```

Tips and edge cases

Use a strict $>$ comparison instead of $\geq$ if intervals are closed and cannot touch at an endpoint. If activities have weights/profits, use weighted interval scheduling DP.

LeetCode matches or related practice: [Non Overlapping Intervals](#)

16. Implement stack operations using an array

Problem: Implement push, pop, peek, and isEmpty for an integer stack. Pop and peek on an empty stack should report an error.

Pattern: LIFO storage with a top index. Maintain the invariant that elements occupy array positions 0 through size-1.

1. Brute force: use the front of an array list as the top

O(n) push/pop, O(1) peek

Inserting and removing at index zero shifts existing values.

```
static class ListStack {
    ArrayList<Integer> a=new ArrayList<>();
    void push(int x){a.add(0,x);}
    int pop(){if(a.isEmpty())throw new NoSuchElementException();return a.remove(0);}
    int peek(){if(a.isEmpty())throw new NoSuchElementException();return a.get(0);}
    boolean isEmpty(){return a.isEmpty();}
}
```

2. Better: fixed-capacity array

O(1) per operation, O(capacity) space

Use size as the next free index. Report overflow when the array is full.

```
static class FixedStack {
    int[] a; int size=0;
    FixedStack(int capacity){a=new int[capacity];}
    void push(int x){if(size==a.length)throw new IllegalStateException("full");a[size++]=x;}
    int pop(){if(size==0)throw new NoSuchElementException();return a[--size];}
    int peek(){if(size==0)throw new NoSuchElementException();return a[size-1];}
    boolean isEmpty(){return size==0;}
}
```

3. Best general-purpose array stack: resize when full

Amortized O(1) push/pop, O(n) space

Double the backing array at capacity; shrink is optional. The top remains at size-1.

```
static class IntStack {
    private int[] a=new int[4]; private int size;
    void push(int x){if(size==a.length)a=Arrays.copyOf(a,a.length*2);a[size++]=x;}
    int pop(){if(size==0)throw new NoSuchElementException();return a[--size];}
    int peek(){if(size==0)throw new NoSuchElementException();return a[size-1];}
    boolean isEmpty(){return size==0;}
    int size(){return size;}
}
```

Tips and edge cases

Distinguish empty-stack underflow from full fixed-capacity overflow. Reset popped array cells only if retaining references; this example stores primitive ints. Java already provides ArrayDeque for production stack use.

GeeksforGeeks: [Open source problem / topic](#)

17. Check whether brackets are balanced

Problem: Given a string containing brackets, return true if every opening bracket has the matching closing type and all brackets close in the correct order. Non-bracket characters are ignored.

Pattern: Stack; the most recently opened bracket must be the next one closed.

1. Brute force: repeatedly remove adjacent matched pairs

$O(n^2)$ time, $O(n)$ space

Delete (), [], or {} pairs until no more are found; the remainder is balanced only if empty.

```
static boolean balancedBrute(String s) {
    StringBuilder x = new StringBuilder(s);
    boolean changed = true;
    while (changed) {
        changed = false;
        for (int i = 0; i + 1 < x.length(); i++) {
            String pair = x.substring(i, i + 2);
            if (pair.equals("(") || pair.equals("[") || pair.equals("{")) {
                x.delete(i, i + 2); changed = true; break;
            }
        }
    }
    for (int i = 0; i < x.length(); i++)
        if ("()[]{}".indexOf(x.charAt(i)) >= 0) return false;
    return true;
}
```

2. Better: count each bracket type

$O(n)$ time, $O(1)$ space; only works for one bracket type

For parentheses only, track balance and reject a negative prefix or nonzero final count.

```
static boolean balancedParenOnly(String s) {
    int balance = 0;
    for (char c : s.toCharArray()) {
        if (c == '(') balance++;
        else if (c == ')' && --balance < 0) return false;
    }
    return balance == 0;
}
```

3. Best: stack opening bracket types

$O(n)$ time, $O(n)$ space

Push openers; each closer must match the stack top, and the stack must be empty at the end.

```
static boolean isBalanced(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else if (c == ')' || c == ']' || c == '}') {
            if (st.isEmpty()) return false;
            char open = st.pop();
            if ((c == ')' && open != '(') || (c == ']' && open != '[') || (c == '}' && open != '{')) return false;
        }
    }
    return st.isEmpty();
}
```

Tips and edge cases

A counter is enough only when there is one bracket type. With multiple bracket types, nesting order matters. Test early closing, mismatched type, leftover opener, and ordinary text.

GeeksforGeeks: [Open source problem / topic](#)

LeetCode matches or related practice: [Valid Parentheses](#) • [Longest Valid Parentheses](#)

18. Circular queue operations

Problem: Implement a fixed-capacity circular queue with enqueue, dequeue, front, and isEmpty. Enqueue on full and dequeue/front on empty should report failure.

Pattern: Ring buffer; head identifies the front, size distinguishes full from empty, and indices wrap with modulo capacity.

1. Brute force: shift all values after dequeue

$O(n)$ dequeue, $O(1)$ enqueue

Store queue contents contiguously and shift the remaining values left when removing the front.

```
static class ShiftQueue {
    int[] a; int size;
    ShiftQueue(int c){a=new int[c];}
    void add(int x){if(size==a.length)throw new IllegalStateException("full");a[size++]=x;}
    int remove(){if(size==0)throw new NoSuchElementException();int
        x=a[0];System.arraycopy(a,1,a,0,--size);return x;}
    int front(){if(size==0)throw new NoSuchElementException();return a[0];}
}
```

2. Better: circular indices with an explicit size

$O(1)$ per operation, $O(\text{capacity})$ space

Advance head and tail modulo capacity; size cleanly separates full from empty.

```
static class RingQueue {
    int[] a; int head=0, size=0;
    RingQueue(int c){if(c<=0)throw new IllegalArgumentException();a=new int[c];}
    boolean offer(int x){if(size==a.length)return false;a[(head+size)%a.length]=x;size++;return
        true;}
    int poll(){if(size==0)throw new NoSuchElementException();int
        x=a[head];head=(head+1)%a.length;size--;return x;}
    int front(){if(size==0)throw new NoSuchElementException();return a[head];}
    boolean isEmpty(){return size==0;}
}
```

3. Best: sentinel-index ring buffer

$O(1)$ per operation, $O(\text{capacity}+1)$ space

Reserve one empty slot. head==tail is empty; advancing tail to head would be full.

```
static class CircularQueue {
    int[] a; int head=0, tail=0;
    CircularQueue(int capacity){if(capacity<=0)throw new IllegalArgumentException();a=new
        int[capacity+1];}
    boolean isEmpty(){return head==tail;}
    boolean isFull(){return (tail+1)%a.length==head;}
    boolean offer(int x){if(isFull())return false;a[tail]=x;tail=(tail+1)%a.length;return true;}
    int poll(){if(isEmpty())throw new NoSuchElementException();int
        x=a[head];head=(head+1)%a.length;return x;}
    int front(){if(isEmpty())throw new NoSuchElementException();return a[head];}
}
```

Tips and edge cases

Handle capacity zero explicitly. Test wraparound after several removes and adds, then test full and empty conditions. A size counter and a reserved-slot design are both correct; do not mix their full tests.

GeeksforGeeks: [Open source problem / topic](#)

19. Noise in the Library (hashing; representative formulation)

Problem: Assumption because the practical list provides only a title: given a collection of lowercase words heard/reported in a library, return the most frequent word; break ties by lexicographically smaller word. Replace this statement if your instructor gave a different version.

Pattern: Hashing and frequency counting; map each token to its count, then select the maximum count with the specified tie rule.

1. Brute force: recount every word for each candidate

$O(n^2 * L)$ time, $O(1)$ extra space beyond input

For every word, scan the full list to count matches; retain the most frequent candidate.

```
static String noiseBrute(String[] words) {
    String best="";int bestCount=0;
    for(String w:words){int c=0;for(String x:words)if(x.equals(w))c++;
        if(c>bestCount||(c==bestCount&&w.compareTo(best)<0)){best=w;bestCount=c;}}
    return best;
}
```

2. Better: sort, then count runs

$O(n \log n * L)$ time, $O(n)$ copy

Sorting groups equal tokens together so each run can be counted once.

```
static String noiseSorted(String[] input) {
    String[] a=input.clone();Arrays.sort(a);String best="";int bestCount=0;
    for(int i=0;i<a.length;){int j=i+1;while(j<a.length&&a[j].equals(a[i]))j++;
        int c=j-i;if(c>bestCount){best=a[i];bestCount=c;}i=j;}
    return best;
}
```

3. Best: HashMap frequency table

$O(n * L)$ expected time, $O(u * L)$ space

Count in one pass, then scan distinct tokens. Since sorted tie-breaking is not needed, compare ties directly.

```
static String mostFrequentWord(String[] words) {
    Map<String,Integer> f=new HashMap<>();
    for(String w:words)f.merge(w,1,Integer::sum);
    String best="";int count=0;
    for(Map.Entry<String,Integer> e:f.entrySet()){
        int c=e.getValue();String w=e.getKey();
        if(c>count||(c==count&&w.compareTo(best)<0)){best=w;count=c;}
    }
    return best;
}
```

Tips and edge cases

The name alone does not specify the input format or expected answer. This chapter uses “most frequent word” as a hashing exercise and makes the tie rule explicit. Normalize case/punctuation only if the original statement requires it.

LeetCode matches or related practice: [Most Common Word](#)

20. Use hashing to balance a scale with given weights

Problem: Given positive integer weights, determine whether they can be split into two groups with equal total weight. Each weight must be placed on exactly one side. Return true if a balanced split exists.

Pattern: Subset-sum reachability; store sums that can be formed so far. HashSet makes the reachable states explicit.

1. Brute force: try every assignment

$O(2^n)$ time, $O(n)$ stack

Assign each weight to the left or right pan, then compare final totals.

```
static boolean balanceBrute(int[] w,int i,int diff) {
    if(i==w.length)return diff==0;
    return balanceBrute(w,i+1,diff+w[i])||balanceBrute(w,i+1,diff-w[i]);
}
```

2. Better: boolean subset-sum table

$O(nS)$ time, $O(S)$ space

If total S is odd, impossible. Otherwise compute which sums through $S/2$ are reachable.

```
static boolean balanceDp(int[] w) {
    long total=0;for(int x:w)total+=x;if((total&1)==1)return false;
    if(total/2>Integer.MAX_VALUE)throw new
        IllegalArgumentException("sum too large for this table");int target=(int)(total/2);
    boolean[] dp=new boolean[target+1];dp[0]=true;
    for(int x:w)for(int s=target;s>=x;s--)dp[s]=dp[s-x];
    return dp[target];
}
```

3. Best for sparse reachable sums: HashSet states

$O(nR)$ expected time, $O(R)$ space

For each weight, snapshot prior sums and add each sum plus that weight. R is the number of distinct reachable sums.

```
static boolean balanceHash(int[] w) {
    long total=0;for(int x:w)total+=x;if((total&1)!=0)return false;long target=total/2;
    Set<Long> sums=new HashSet<>();sums.add(0L);
    for(int x:w){Set<Long> next=new HashSet<>(sums);for(long s:sums)next.add(s+x);sums=next;}
    return sums.contains(target);
}
```

Tips and edge cases

This is the equal-partition interpretation of “balance a scale.” If weights may be left unused or placed on either pan, the state must track signed differences instead. Use long when total weight may exceed int.

LeetCode matches or related practice: [Partition Equal Subset Sum](#)

21. Find the winner in a voting system using hashing

Problem: Given candidate names for all votes, return the candidate with the most votes. For ties, choose the lexicographically smallest name.

Pattern: HashMap frequency counting; update the current winner by count, then tie-break rule.

1. Brute force: count each candidate by rescanning votes

$O(n^2 * L)$ time

For every distinct candidate, count its occurrences by scanning the full ballot list.

```
static String winnerBrute(String[] votes) {
    String best="";int max=0;
    for(String c:votes){int n=0;for(String v:votes)if(v.equals(c))n++;
        if(n>max||(n==max&&c.compareTo(best)<0)){max=n;best=c;}}
    return best;
}
```

2. Better: sort and count candidate runs

$O(n \log n * L)$ time, $O(n)$ copy

Sort candidate names so equal votes are adjacent; a run is one candidate's total.

```
static String winnerSorted(String[] votes) {
    String[] a=votes.clone();Arrays.sort(a);String best="";int max=0;
    for(int i=0;i<a.length;){int j=i+1;while(j<a.length&&a[j].equals(a[i]))j++;
        if(j-i>max){max=j-i;best=a[i];i=j;}
    }
    return best;
}
```

3. Best: one-pass HashMap count

$O(n * L)$ expected time, $O(u * L)$ space

Count every vote, then scan the candidate totals once using the same tie rule.

```
static String electionWinner(String[] votes) {
    Map<String,Integer> count=new HashMap<>();
    for(String v:votes)count.merge(v,1,Integer::sum);
    String best="";int max=0;
    for(var e:count.entrySet())if(e.getValue()>max||(e.getValue()==max&&e.getKey().compareTo(best)<0)){
        best=e.getKey();max=e.getValue();}
    return best;
}
```

Tips and edge cases

Confirm the tie rule in the assignment; another common convention is first candidate in input order. Return a defined value for an empty vote list if that case is allowed.

LeetCode matches or related practice: [Majority Element](#)

22. Generate all permutations of a string

Problem: Given a string, print or return every distinct permutation. Duplicate input letters should not produce duplicate output strings.

Pattern: Backtracking; choose one unused character for the next position, recurse, then undo the choice.

1. Brute force: generate permutations then deduplicate

$O(n! * n)$ time, $O(n! * n)$ output space

Generate every positional permutation, then insert outputs into a set. Repeated letters cause duplicate work.

```
static Set<String> permuteSet(String s) {
    Set<String> out=new HashSet<>();permuteAll(s.toCharArray(),0,out);return out;
}
static void permuteAll(char[] a,int i,Set<String> out) {
    if(i==a.length){out.add(new String(a));return;}
    for(int j=i;j<a.length;j++){char
        t=a[i];a[i]=a[j];a[j]=t;permuteAll(a,i+1,out);t=a[i];a[i]=a[j];a[j]=t;}
}
```

2. Better: backtrack with a used array

$O(n * n!)$ time, $O(n)$ recursion plus outputs

Build the output one position at a time. This avoids mutating the input but still duplicates branches for repeated characters.

```
static List<String> permutationsUsed(String s) {
    char[] a=s.toCharArray();boolean[] used=new boolean[a.length];List<String> out=new
    ArrayList<>();
    permuteUsed(a,used,new StringBuilder(),out);return out;
}
static void permuteUsed(char[] a,boolean[] used,StringBuilder p,List<String> out) {
    if(p.length()==a.length){out.add(p.toString());return;}
    for(int
        i=0;i<a.length;i++){if(!used[i]){used[i]=true;p.append(a[i]);permuteUsed(a,used,p,out);p.setLength(p.length()-1);used[i]=false;}
    }
}
```

3. Best: sort and skip duplicate choices

$O(n * \text{numberOfDistinctPermutations})$ time

Sort characters first. At each recursion depth, skip a character equal to the previous one when the previous copy is not currently chosen.

```
static List<String> permutations(String s) {
    char[] a=s.toCharArray();Arrays.sort(a);boolean[] used=new boolean[a.length];
    List<String> out=new ArrayList<>();permuteUnique(a,used,new StringBuilder(),out);return out;
}
static void permuteUnique(char[] a,boolean[] used,StringBuilder p,List<String> out) {
    if(p.length()==a.length){out.add(p.toString());return;}
    for(int i=0;i<a.length;i++){
        if(used[i]||i>0&&a[i]==a[i-1]&&!used[i-1])continue;
        used[i]=true;p.append(a[i]);permuteUnique(a,used,p,out);p.setLength(p.length()-1);used[i]=false;
    }
}
```

Tips and edge cases

There are $n!$ outputs when all characters differ, so output cost is unavoidable. For duplicate characters, the number of unique permutations is $n!$ divided by each repeated-character factorial.

LeetCode matches or related practice: [Permutations](#) • [Permutations II](#)

23. Tower of Hanoi using recursion

Problem: Move n disks from source peg A to destination peg C using auxiliary peg B. Move one disk at a time and never place a larger disk on a smaller one. Print each move.

Pattern: Recursive decomposition: move $n-1$ disks aside, move the largest disk, then move $n-1$ onto it.

1. Brute force for tiny n : breadth-first search of legal states

Exponential state space (3^n states)

Represent each disk by its peg and search legal moves until all disks reach the target. This is useful only to verify tiny examples.

```
static List<String> hanoiBfs(int n) {
    String start="A".repeat(n), goal="C".repeat(n); Queue<String> q=new ArrayDeque<>();
    Map<String,String> parent=new HashMap<>(); Map<String,String> move=new
        HashMap<>(); q.add(start); parent.put(start,null);
    while(!q.isEmpty()){
        String state=q.remove(); if(state.equals(goal)) break;
        for(char from='A'; from<='C'; from++){
            int disk=-1; for(int i=0; i<n; i++){ if(state.charAt(i)==from){ disk=i; break; }
            if(disk<0) continue;
            for(char to='A'; to<='C'; to++){ if(to!=from){ int top=-1; for(int
                i=0; i<n; i++){ if(state.charAt(i)==to){ top=i; break; }
                if(top<0 || disk<top){ char[] z=state.toCharArray(); z[disk]=to; String next=new
                    String(z); if(!parent.containsKey(next)){ parent.put(next,state); move.put(next,(char)('1'+disk)+" "+from+"->" +to); q.add(next); }
            }
        }
    }
    List<String> ans=new ArrayList<>(); for(String
        s=goal; parent.get(s)!=null; s=parent.get(s)) ans.add(move.get(s)); Collections.reverse(ans); return ans;
}
```

2. Better: recursive recurrence

Exactly $2^n - 1$ moves; $O(n)$ stack

Apply the recurrence directly: solve $n-1$, move the largest disk, solve $n-1$.

```
static void hanoi(int n, char from, char aux, char to, List<String> out) {
    if(n==0) return;
    hanoi(n-1, from, to, aux, out);
    out.add("Move disk "+n+" from "+from+" to "+to);
    hanoi(n-1, aux, from, to, out);
}
```

3. Best: the same recurrence with streamed output

Optimal: $O(2^n)$ time because every move must be printed; $O(n)$ stack

Avoid storing all moves when the output can be emitted immediately. No solution can print all moves asymptotically faster than the output size.

```
static void hanoiPrint(int n, char from, char aux, char to) {
    if(n==0) return;
    hanoiPrint(n-1, from, to, aux);
    System.out.println("Move disk "+n+" from "+from+" to "+to);
    hanoiPrint(n-1, aux, from, to);
}
```

Tips and edge cases

The optimal move count is $2^n - 1$. The BFS code assumes n is small and Java string repeat is available (Java 11+). For the requested recursive practical, focus on the recursive version and its base case.

24. Move a robot through a grid using backtracking

Problem: Assumption: given a rectangular grid where 0 is open and 1 is blocked, find one path from the top-left cell to the bottom-right cell using four-direction moves. Return an empty path if unreachable.

Pattern: Backtracking on a grid; mark a cell visited on the current path, explore neighbors, then unmark when retreating.

1. Brute force: enumerate move strings up to a simple-path limit

Exponential in number of cells

Try each four-direction move sequence, reject out-of-grid, blocked, repeated-cell, or non-destination sequences.

```
static boolean robotBrute(int[][] g, int r, int c, int[][] seen) {
    int m = g.length, n = g[0].length;
    if (r < 0 || c < 0 || r >= m || c >= n || g[r][c] == 1 || seen[r][c] == 1) return false;
    if (r == m - 1 && c == n - 1) return true;
    seen[r][c] = 1;
    boolean ok = robotBrute(g, r + 1, c, seen) || robotBrute(g, r, c + 1, seen) || robotBrute(g, r - 1, c, seen) || robotBrute(g, r, c - 1, seen);
    seen[r][c] = 0; return ok;
}
```

2. Better: DFS backtracking with visited cells

$O(V+E)$ for reachability, $O(V)$ space

Avoid revisiting cells on the current route. Record a direction when taking a successful branch.

```
static String robotPath(int[][] g) {
    if (g.length == 0 || g[0].length == 0) return "";
    StringBuilder path = new StringBuilder();
    boolean[][] seen = new boolean[g.length][g[0].length];
    return robotDfs(g, 0, 0, seen, path) ? path.toString() : "";
}

static boolean robotDfs(int[][] g, int r, int c, boolean[][] seen, StringBuilder p) {
    int m = g.length, n = g[0].length;
    if (r < 0 || c < 0 || r >= m || c >= n || g[r][c] != 0 || seen[r][c]) return false;
    if (r == m - 1 && c == n - 1) return true;
    seen[r][c] = true;
    int[] dr = {1, 0, 0, -1}, dc = {0, 1, -1, 0};
    char[] d = {'D', 'R', 'L', 'U'};
    for (int i = 0; i < 4; i++) {
        p.append(d[i]);
        if (robotDfs(g, r + dr[i], c + dc[i], seen, p)) return true;
        p.setLength(p.length() - 1);
        seen[r][c] = false;
    }
    return false;
}
```

3. Best for shortest route: BFS with parent directions

$O(V+E)$ time, $O(V)$ space

If “move the robot” means find a shortest path, BFS visits cells by distance and stores each cell’s predecessor direction.

```
static String robotShortest(int[][] g) {
    int m = g.length, n = g[0].length;
    if (m == 0 || n == 0 || g[0][0] != 0 || g[m - 1][n - 1] != 0) return "";
    int[][] pr = new int[m][n], pc = new int[m][n];
    for (int[] x : pr) Arrays.fill(x, -1);
    int[] dr = {1, 0, 0, -1}, dc = {0, 1, -1, 0};
    char[] d = {'D', 'R', 'L', 'U'};
    Queue<int[]> q = new ArrayDeque<>();
    q.add(new int[]{0, 0});
    pr[0][0] = 0;
    while (!q.isEmpty()) {
        int[] x = q.remove();
        for (int i = 0; i < 4; i++) {
            int r = x[0] + dr[i], c = x[1] + dc[i];
            if (r >= 0 && r < m && c >= 0 && c < n && g[r][c] == 0 && pr[r][c] == -1) {
                pr[r][c] = x[0] + 1;
                pc[r][c] = x[1] + 1;
                q.add(new int[]{r, c});
            }
        }
    }
    if (pr[m - 1][n - 1] == -1) return "";
    StringBuilder ans = new StringBuilder();
    int r = m - 1, c = n - 1;
    while (r != 0 || c != 0) {
        int rr = pr[r][c] - 1, cc = pc[r][c] - 1;
        for (int i = 0; i < 4; i++) {
            if (rr + dr[i] == r && cc + dc[i] == c) {
                ans.append(d[i]);
                r = rr;
                c = cc;
            }
        }
    }
    return ans.reverse().toString();
}
```

Tips and edge cases

The screenshot does not say if a shortest path is required or which moves are legal. The first two methods find a path with backtracking; BFS returns a shortest path for unit-cost moves. Validate rectangular input before indexing.

LeetCode matches or related practice: [Shortest Path In Binary Matrix](#)

25. Solve Sudoku using backtracking

Problem: Fill a 9x9 board of digits 1 through 9 and empty cells represented by 0 so every row, column, and 3x3 box contains each digit exactly once.

Pattern: Constraint backtracking; choose an empty cell, try legal digits, recurse, and undo on failure. Choosing the most constrained empty cell reduces branching.

1. Brute force: try digits and check only at completion

Up to 9^E assignments for E empty cells

Assign any digit to each empty square, then test whether the completed board obeys Sudoku rules.

```
static boolean sudokuBrute(int[][] b, int pos) {
    if(pos==81) return validBoard(b);
    int r=pos/9, c=pos%9; if(b[r][c]!=0) return sudokuBrute(b, pos+1);
    for(int d=1; d<=9; d++){ b[r][c]=d; if(sudokuBrute(b, pos+1)) return true; b[r][c]=0; }
    return false;
}

static boolean validBoard(int[][] b) {
    for(int r=0; r<9; r++){ for(int c=0; c<9; c++){ if(b[r][c]!=0){ int x=b[r][c]; b[r][c]=0; boolean
        ok=sudokuCanPlace(b, r, c, x); b[r][c]=x; if(!ok) return false; }
    }
    return true;
}

static boolean sudokuCanPlace(int[][] b, int r, int c, int d) {
    for(int i=0; i<9; i++){ if(b[r][i]==d || b[i][c]==d) return false;
    int br=r/3*3, bc=c/3*3; for(int i=br; i<br+3; i++){ for(int j=bc; j<bc+3; j++){ if(b[i][j]==d) return
        false;
    }
    return true;
}
}
```

2. Better: validate each placement immediately

Exponential worst case, $O(81)$ recursion

Reject a digit as soon as it conflicts with a row, column, or box; do not wait for a completed board.

```
static boolean sudokuBacktrack(int[][] b, int pos) {
    if(pos==81) return true; int r=pos/9, c=pos%9; if(b[r][c]!=0) return sudokuBacktrack(b, pos+1);
    for(int d=1; d<=9; d++){ if(sudokuCanPlace(b, r, c, d)){ b[r][c]=d; if(sudokuBacktrack(b, pos+1)) return
        true; b[r][c]=0; }
    return false;
}
}
```

3. Best practical search: bitmasks plus minimum-remaining-values

Exponential worst case; fast for typical 9x9 boards

Maintain row/column/box masks and select the empty cell with the fewest legal digits before recursing.

```
static boolean sudoku(int[][] b) {
    int[] row=new int[9], col=new int[9], box=new int[9];
    for(int r=0; r<9; r++){ for(int c=0; c<9; c++){ if(b[r][c]!=0){ int
        bit=1<<b[r][c], z=(r/3)*3+c/3; row[r]=bit; col[c]=bit; box[z]=bit; }
    }
    return sudokuSearch(b, row, col, box);
}

static boolean sudokuSearch(int[][] b, int[] row, int[] col, int[] box) {
    int br=-1, bc=-1, opts=0, best=10;
    for(int r=0; r<9; r++){ for(int c=0; c<9; c++){ if(b[r][c]==0){ int
        mask=~(row[r]|col[c]|box[(r/3)*3+c/3])&0x3FE; int
        n=Integer.bitCount(mask); if(n<best){ best=n; br=r; bc=c; opts=mask; }
    }
    if(br<0) return true; if(best==0) return false; int z=(br/3)*3+bc/3;
    for(int mask=opts; mask!=0; mask&=mask-1){ int
        bit=mask&~mask, d=Integer.numberOfTrailingZeros(bit); b[br][bc]=d; row[br]=bit; col[bc]=bit; box[z]=bit;
        if(sudokuSearch(b, row, col, box)) return
            true; b[br][bc]=0; row[br]^=bit; col[bc]^=bit; box[z]^=bit; }
    return false;
}
}
```

Tips and edge cases

The bitmask solver assumes a valid starting board. If initial clues may conflict, validate them while building masks. A successful recursion leaves the solved board in place; a failed one restores zeros.

LeetCode matches or related practice: [Sudoku Solver](#)

26. Find all paths for Rat in a Maze

Problem: Given an $n \times n$ maze where 1 is open and 0 is blocked, return every path from (0,0) to (n-1,n-1). A move may go up, down, left, or right; a path may not revisit a cell.

Pattern: Backtracking over a grid; maintain a path-local visited mark, append a move, recurse, and undo both when returning.

1. Brute force: copy visited state at every branch

Exponential in open cells; $O(n^2)$ state copied per branch

Explore all simple paths while cloning the visited grid for each recursive choice.

```
static void mazeCopy(int[][] m, int r, int c, boolean[][] seen, StringBuilder p, List<String> out) {
    int n=m.length;if(r<0||c<0||r>=n||c>=n||m[r][c]==0||seen[r][c])return;
    if(r==n-1&&c==n-1){out.add(p.toString());return;}
    boolean[][] next=new boolean[n][n];for(int
        i=0;i<n;i++)next[i]=seen[i].clone();next[r][c]=true;
    int[] dr={1,0,0,-1},dc={0,-1,1,0};char[] d={'D','L','R','U'};
    for(int
        i=0;i<4;i++){p.append(d[i]);mazeCopy(m,r+dr[i],c+dc[i],next,p,out);p.setLength(p.length()-1);}
}
```

2. Better: in-place visited backtracking

$O(\text{number of simple paths} \times \text{path length})$, $O(n^2)$ visited

Mark a cell while it is on the current path, then unmark it so other paths can use it.

```
static List<String> ratMaze(int[][] m) {
    List<String> out=new
        ArrayList<>();if(m.length==0||m[0].length!=m.length||m[0][0]==0||m[m.length-1][m.length-1]==0)return out;
    mazeDfs(m,0,0,new boolean[m.length][m.length],new
        StringBuilder(),out);Collections.sort(out);return out;
}
static void mazeDfs(int[][] m, int r, int c, boolean[][] seen, StringBuilder p, List<String> out) {
    int n=m.length;if(r<0||c<0||r>=n||c>=n||m[r][c]==0||seen[r][c])return;
    if(r==n-1&&c==n-1){out.add(p.toString());return;}seen[r][c]=true;
    int[] dr={1,0,0,-1},dc={0,-1,1,0};char[] d={'D','L','R','U'};
    for(int
        i=0;i<4;i++){p.append(d[i]);mazeDfs(m,r+dr[i],c+dc[i],seen,p,out);p.setLength(p.length()-1);seen[r][c]=false;}
}
```

3. Best memory use: mark and restore the maze cells

Same output-sensitive time; $O(\text{path length})$ extra space

Use a temporary marker in the grid instead of a second $n \times n$ visited array; restore every cell during backtracking.

```
static List<String> allMazePaths(int[][] m) {
    List<String> ans=new ArrayList<>();int
        n=m.length;if(n==0||m[0].length!=n||m[0][0]!=1||m[n-1][n-1]!=1)return ans;
    mazeInPlace(m,0,0,new StringBuilder(),ans);Collections.sort(ans);return ans;
}
static void mazeInPlace(int[][] m, int r, int c, StringBuilder p, List<String> out) {
    int n=m.length;if(r<0||c<0||r>=n||c>=n||m[r][c]!=1)return;
    if(r==n-1&&c==n-1){out.add(p.toString());return;}m[r][c]=2;
    int[] dr={1,0,0,-1},dc={0,-1,1,0};char[] d={'D','L','R','U'};
    for(int
        i=0;i<4;i++){p.append(d[i]);mazeInPlace(m,r+dr[i],c+dc[i],p,out);p.setLength(p.length()-1);m[r][c]=1;}
}
```

Tips and edge cases

The three approaches share exponential output cost because all simple paths may be requested. If the course defines only right/down moves, remove the U and L directions and the visited array is unnecessary.

LeetCode matches or related practice: [Unique Paths](#)

27. Print all binary strings of length n

Problem: Given $n \geq 0$, print every string of exactly n binary digits in lexicographic order.

Pattern: Backtracking over a fixed-length output buffer; each position has two choices.

1. Brute force: count from 0 to $2^n - 1$ and convert

$O(n \cdot 2^n)$ time, $O(n)$ temporary space

Convert each integer to its n -bit representation, including leading zeroes.

```
static List<String> binaryBrute(int n) {
    if(n<0||n>30)throw new IllegalArgumentException();List<String> out=new
        ArrayList<>();if(n==0){out.add("");return out;}
    for(int
        x=0;x<(1<n);x++)out.add(String.format("%"+n+"s",Integer.toBinaryString(x)).replace(' ', '0'));
    return out;
}
```

2. Better: recursive string construction

$O(n \cdot 2^n)$ time, $O(n)$ stack plus output

Append 0 then 1 at every position.

```
static List<String> binaryRecursive(int n) {
    List<String> out=new ArrayList<>();buildBits(n,new StringBuilder(),out);return out;
}
static void buildBits(int n,StringBuilder s,List<String> out) {
    if(s.length()==n){out.add(s.toString());return;}
    s.append('0');buildBits(n,s,out);s.setLength(s.length()-1);
    s.append('1');buildBits(n,s,out);s.setLength(s.length()-1);
}
```

3. Best: reuse a char buffer and stream results

$O(2^n)$ output time, $O(n)$ stack/buffer

Avoid constructing and retaining all results when the practical only asks to print them.

```
static void printBinary(int n) { printBits(new char[n],0); }
static void printBits(char[] a,int i) {
    if(i==a.length){System.out.println(new String(a));return;}
    a[i]='0';printBits(a,i+1);a[i]='1';printBits(a,i+1);
}
```

Tips and edge cases

There are 2^n outputs, so exponential runtime is unavoidable when printing every string. The integer-conversion version is limited by Java int width; recursion works for larger n subject to stack/output limits.

LeetCode matches or related practice: [Letter Combinations Of A Phone Number](#)

28. Solve N-Queens using backtracking

Problem: Place N queens on an N x N chessboard so no two queens share a row, column, or diagonal. Return or print every valid board.

Pattern: Backtracking by rows; track occupied columns and diagonals to reject attacking placements in constant time.

1. Brute force: enumerate column choices then validate

$O(N^N)$ candidates

Assign any column to every row, then check column and diagonal conflicts only after a full board is formed.

```
static int queensBrute(int n,int row,int[] col) {
    if(row==n)return validQueens(col,n)?1:0;int count=0;
    for(int c=0;c<n;c++){col[row]=c;count+=queensBrute(n,row+1,col);}return count;
}
static boolean validQueens(int[] q,int n) {
    for(int i=0;i<n;i++)for(int j=0;j<i;j++)if(q[i]==q[j]||Math.abs(q[i]-q[j])==i-j)return
        false;
    return true;
}
```

2. Better: row-by-row backtracking with scan checks

$O(N!)$ worst case

Only place a queen in a column with no earlier queen on the same column or diagonal.

```
static int queensScan(int n) { return placeQueenScan(n,0,new int[n]); }
static int placeQueenScan(int n,int r,int[] q) {
    if(r==n)return 1;int count=0;
    for(int c=0;c<n;c++){boolean ok=true;for(int
        p=0;p<r;p++){if(q[p]==c||Math.abs(q[p]-c)==r-p){ok=false;break;}}
    if(ok){q[r]=c;count+=placeQueenScan(n,r+1,q);}}
    return count;
}
```

3. Best for small N: bitmask columns and diagonals

Exponential search with $O(1)$ conflict checks

Represent columns and two diagonal families as bit sets; branch only over currently available columns.

```
static long nQueens(int n) {
    if(n<0||n>15)throw new IllegalArgumentException("bitmask demo supports n <= 15");
    return queensBits(n,0,0,0,(1<n)-1);
}
static long queensBits(int n,int cols,int down,int up,int all) {
    if(cols==all)return 1;
    int free=all&~(cols|down|up);long count=0;
    while(free!=0){int
        bit=free&-free;free-=bit;count+=queensBits(n,cols|bit,((down|bit)<<1)&all,(up|bit)>>>1,all);}
    return count;
}
```

Tips and edge cases

Bitmask diagonals shift as recursion moves one row down. The count fits long only for practical N values; printing board layouts needs storing the chosen column per row.

LeetCode matches or related practice: [N Queens](#)

29. Lowest common ancestor in a binary search tree

Problem: Given a BST and two node values p and q that exist in the tree, return their lowest common ancestor.

Pattern: Use the BST ordering; if both values are smaller, go left; if both are larger, go right; otherwise the current node is the split point.

1. Brute force: general binary-tree postorder search

$O(n)$ time, $O(h)$ recursion

Find both targets in an arbitrary binary tree; the node whose left and right subtrees contain the two targets is the answer.

```
static TreeNode lcaGeneral(TreeNode root,TreeNode p,TreeNode q) {
    if(root==null||root==p||root==q)return root;
    TreeNode l=lcaGeneral(root.left,p,q),r=lcaGeneral(root.right,p,q);
    return l!=null&&r!=null?root:(l!=null?l:r);
}
```

2. Better: store root-to-node paths

$O(h)$ time, $O(h)$ extra space

Build both paths using BST comparisons and return their last common node.

```
static List<TreeNode> bstPath(TreeNode root,int x) {
    List<TreeNode> p=new
        ArrayList<>();while(root!=null){p.add(root);if(root.val==x)break;root=x<root.val?root.left:root.right;}return p;
}
static TreeNode lcaPaths(TreeNode root,int p,int q) {
    List<TreeNode> a=bstPath(root,p),b=bstPath(root,q);TreeNode ans=null;
    for(int i=0;i<Math.min(a.size(),b.size())&&a.get(i)==b.get(i);i++)ans=a.get(i);return ans;
}
```

3. Best: walk to the split point

$O(h)$ time, $O(1)$ extra space

The first node between p and q (inclusive) is their LCA.

```
static TreeNode lcaBst(TreeNode root,int p,int q) {
    while(root!=null){
        if(p<root.val&&q<root.val)root=root.left;
        else if(p>root.val&&q>root.val)root=root.right;
        else return root;
    }
    return null;
}
```

Tips and edge cases

This assumes both values exist; otherwise the split-point walk may return a node despite a missing target. Validate membership first if the interface permits absent keys. For a general binary tree, use the first approach.

LeetCode matches or related practice: [Lowest Common Ancestor Of A Binary Search Tree](#)

30. Find the kth smallest element in a BST

Problem: Given a BST with distinct keys and $1 \leq k \leq \text{node count}$, return its kth smallest value.

Pattern: BST inorder traversal visits values in ascending order. Stop once k values have been visited.

1. Brute force: collect and sort all values

$O(n \log n)$ time, $O(n)$ space

Traverse every node into a list, sort it, then return position k-1.

```
static int kthSort(TreeNode root,int k) {
    List<Integer>a=new ArrayList<>();collect(root,a);Collections.sort(a);return a.get(k-1);
}
static void collect(TreeNode
    n,List<Integer>a){if(n==null)return;a.add(n.val);collect(n.left,a);collect(n.right,a);}
```

2. Better: recursive inorder with early stop

$O(h+k)$ average traversal, $O(h)$ stack

Visit left subtree, current node, then right subtree; stop recursing when k reaches zero.

```
static class KState { int k,answer;KState(int k){this.k=k;} }
static int kthRecursive(TreeNode root,int k) { KState s=new KState(k);inorderK(root,s);return
    s.answer; }
static void inorderK(TreeNode n,KState s) {
    if(n==null||s.k==0)return;inorderK(n.left,s);if(s.k>0&&--s.k==0)s.answer=n.val;inorderK(n.right,s);
}
```

3. Best: iterative inorder stack

$O(h+k)$ time, $O(h)$ space

Push the left spine, pop the next smallest, then explore its right subtree.

```
static int kthSmallest(TreeNode root,int k) {
    Deque<TreeNode> st=new ArrayDeque<>();TreeNode cur=root;
    while(cur!=null||!st.isEmpty()){
        while(cur!=null){st.push(cur);cur=cur.left;}
        cur=st.pop();if(--k==0)return cur.val;cur=cur.right;
    }
    throw new IllegalArgumentException("k out of range");
}
```

Tips and edge cases

Validate k before indexing if the tree size is known. If many kth queries are made between inserts, augment each node with subtree sizes so each query takes $O(h)$.

LeetCode matches or related practice: [Kth Smallest Element In A Bst](#)

31. Perform preorder, inorder, and postorder traversals

Problem: Given the root of a binary tree, return its preorder, inorder, and postorder value lists.

Pattern: Depth-first traversal; the order is determined by when the node is recorded relative to its left and right subtrees.

1. Brute force: run three separate recursive traversals

O(n) total time, O(h) stack per traversal

Each traversal visits every node once, but walks the tree three times.

```
static List<Integer> preorder(TreeNode n){List<Integer> a=new ArrayList<>();pre(n,a);return a;}
static void pre(TreeNode
    n,List<Integer>a){if(n==null)return;a.add(n.val);pre(n.left,a);pre(n.right,a);}
static List<Integer> inorder(TreeNode n){List<Integer>a=new ArrayList<>();in(n,a);return a;}
static void in(TreeNode
    n,List<Integer>a){if(n==null)return;in(n.left,a);a.add(n.val);in(n.right,a);}
static List<Integer> postorder(TreeNode n){List<Integer>a=new ArrayList<>();post(n,a);return a;}
static void post(TreeNode
    n,List<Integer>a){if(n==null)return;post(n.left,a);post(n.right,a);a.add(n.val);}
```

2. Better: iterative traversal stacks

O(n) time, O(h) to O(n) stack space

Use an explicit stack rather than recursive calls. This avoids call-stack overflow on a very skewed tree.

```
static List<Integer> inorderIter(TreeNode root) {
    List<Integer>a=new ArrayList<>();Deque<TreeNode>s=new ArrayDeque<>();TreeNode c=root;
    while(c!=null||!s.isEmpty()){while(c!=null){s.push(c);c=c.left;}c=s.pop();a.add(c.val);c=c.right;}return a;
}
static List<Integer> postorderIter(TreeNode root) {
    List<Integer>a=new ArrayList<>();if(root==null)return a;Deque<TreeNode>s=new
    ArrayDeque<>();s.push(root);
    while(!s.isEmpty()){TreeNode
        n=s.pop();a.add(n.val);if(n.left!=null)s.push(n.left);if(n.right!=null)s.push(n.right);}
    Collections.reverse(a);return a;
}
```

3. Best: one iterative pass for all three orders

O(n) time, O(h) stack

Keep a stage per stack frame: record preorder before left, inorder between children, postorder after right.

```
static class Orders {List<Integer> pre=new ArrayList<>(),in=new ArrayList<>(),post=new
    ArrayList<>();}
static class Frame {TreeNode n;int state;Frame(TreeNode n){this.n=n;}}
static Orders allOrders(TreeNode root) {
    Orders out=new Orders();if(root==null)return out;Deque<Frame>s=new ArrayDeque<>();s.push(new
    Frame(root));
    while(!s.isEmpty()){
        Frame f=s.peek();
        if(f.state==0){out.pre.add(f.n.val);f.state=1;if(f.n.left!=null)s.push(new
            Frame(f.n.left));}
        else if(f.state==1){out.in.add(f.n.val);f.state=2;if(f.n.right!=null)s.push(new
            Frame(f.n.right));}
        else {out.post.add(f.n.val);s.pop();}
    }
    return out;
}
```

Tips and edge cases

Use a consistent `TreeNode` type with `int val`, `left`, and `right` fields (defined in the tree-practicals note). For an empty tree all three lists are empty.

LeetCode matches or related practice: [Binary Tree Preorder Traversal](#) • [Binary Tree Inorder Traversal](#) • [Binary Tree Postorder Traversal](#)

32. Find the height of a binary tree

Problem: Return the height measured in nodes on the longest root-to-leaf path. An empty tree has height 0 and a leaf has height 1.

Pattern: Tree recursion; $\text{height}(\text{node}) = 1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$.

1. Brute force: recompute subtree heights at every node

$O(n^2)$ worst case, $O(h)$ stack

For each node, independently compute both subtree heights, then recurse again into its children.

```
static int heightSlow(TreeNode n) {
    if(n==null)return 0;
    return 1+Math.max(heightSlow(n.left),heightSlow(n.right));
}
static int heightRepeated(TreeNode n) {
    if(n==null)return 0;
    int left=heightSlow(n.left),right=heightSlow(n.right);
    heightRepeated(n.left);heightRepeated(n.right);
    return 1+Math.max(left,right);
}
```

2. Better: one postorder recursive pass

$O(n)$ time, $O(h)$ call stack

Each node returns its height once.

```
static int treeHeight(TreeNode root) {
    if(root==null)return 0;
    return 1+Math.max(treeHeight(root.left),treeHeight(root.right));
}
```

3. Best for very deep trees: level-order BFS

$O(n)$ time, $O(\text{width})$ queue

Count how many complete levels are in the queue. This avoids recursion depth on a skewed tree.

```
static int treeHeightBfs(TreeNode root) {
    if(root==null)return 0;Queue<TreeNode> q=new ArrayDeque<>();q.add(root);int h=0;
    while(!q.isEmpty()){int size=q.size();h++;while(size-->0){TreeNode
        n=q.remove();if(n.left!=null)q.add(n.left);if(n.right!=null)q.add(n.right);}}
    return h;
}
```

Tips and edge cases

Check the course convention: some texts define height as edges, giving empty=-1 and leaf=0. This chapter uses node count. Recursive and BFS methods are both $O(n)$; BFS is safer when the tree may be extremely deep.

LeetCode matches or related practice: [Maximum Depth Of Binary Tree](#)

33. Find the running maximum and minimum in a data stream

Problem: Values arrive one at a time. After each insertion, report the smallest and largest values seen so far.

Pattern: Streaming invariant; retain the extrema of the prefix already processed.

1. Brute force: rescan all values after each arrival

$O(n^2)$ total time, $O(n)$ stored values

Append the new value and recompute min and max across the full prefix.

```
static List<int[]> extremaBrute(int[] stream) {
    List<Integer> seen=new ArrayList<>();List<int[]> out=new ArrayList<>();
    for(int x:stream){seen.add(x);int lo=Integer.MAX_VALUE,hi=Integer.MIN_VALUE;for(int
        v:seen){lo=Math.min(lo,v);hi=Math.max(hi,v);}out.add(new int[]{lo,hi});}
    return out;
}
```

2. Better: maintain min-heap and max-heap

$O(\log n)$ per insert, $O(n)$ space

Insert each value into both priority queues. Their tops give the extrema; this supports richer later queries too.

```
static List<int[]> extremaHeaps(int[] stream) {
    PriorityQueue<Integer> min=new PriorityQueue<>(),max=new
        PriorityQueue<>(Comparator.reverseOrder());
    List<int[]> out=new ArrayList<>();for(int x:stream){min.add(x);max.add(x);out.add(new
        int[]{min.peek(),max.peek()});}return out;
}
```

3. Best for only min and max: two running values

$O(1)$ per insertion, $O(1)$ state

Compare each arriving value with the extrema seen so far.

```
static List<int[]> runningExtrema(int[] stream) {
    List<int[]> out=new ArrayList<>();if(stream.length==0)return out;int
        lo=stream[0],hi=stream[0];
    for(int x:stream){lo=Math.min(lo,x);hi=Math.max(hi,x);out.add(new int[]{lo,hi});}return out;
}
```

Tips and edge cases

This assumes “continuous stream” means all values seen so far. If the assignment means a sliding window of the last k values, use monotonic deques instead; ordinary heaps need lazy deletion or indexed removal.

LeetCode matches or related practice: [Sliding Window Maximum](#)

34. Sort an array using heap sort

Problem: Sort an integer array in ascending order using a binary heap.

Pattern: Heap invariant; build a max heap, repeatedly move the maximum to the end, and restore the heap in the remaining prefix.

1. Brute force: insertion sort

$O(n^2)$ time, $O(1)$ space

Insert each value into its correct place in the sorted prefix.

```
static void insertionSortHeapBaseline(int[] a) {
    for(int i=1;i<a.length;i++){int x=a[i],j=i-1;while(j>=0&& a[j]>x){a[j+1]=a[j];j--;}a[j+1]=x;}
}
```

2. Better: use a library priority queue

$O(n \log n)$ time, $O(n)$ extra space

Insert all values into a max heap, then remove them from largest to smallest into the array from right to left.

```
static void heapSortQueue(int[] a) {
    PriorityQueue<Integer> q=new PriorityQueue<>(Comparator.reverseOrder());for(int
    x:a)q.add(x);
    for(int i=a.length-1;i>=0;i--)a[i]=q.remove();
}
```

3. Best: in-place heap sort

$O(n \log n)$ time, $O(1)$ extra space

Build a max heap bottom-up in $O(n)$, then repeatedly swap the root with the last live element and sift down.

```
static void heapSort(int[] a) {
    int n=a.length;for(int i=n/2-1;i>=0;i--)sink(a,i,n);
    for(int end=n-1;end>0;end--){int t=a[0];a[0]=a[end];a[end]=t;sink(a,0,end);}
}
static void sink(int[] a,int i,int size) {
    while(true){int
        l=2*i+1,r=l+1,b=i;if(l<size&&a[l]>a[b])b=l;if(r<size&&a[r]>a[b])b=r;if(b==i)return;
        int t=a[i];a[i]=a[b];a[b]=t;i=b;}
}
```

Tips and edge cases

Heap sort is in-place and worst-case $O(n \log n)$, but it is not stable. Use long index arithmetic if heap sizes can approach int limits. In Java code, arrays are bounded by int indexing.

LeetCode matches or related practice: [Sort An Array](#)

35. Check whether a binary tree is a max heap

Problem: Return true if a binary tree is both complete and obeys the max-heap order: every parent value is at least each child value.

Pattern: Validate two independent conditions: complete-tree shape and parent-to-child ordering.

1. Brute force: recheck each subtree

$O(n^2)$ worst case, $O(h)$ stack

At each node, scan its subtree to check shape and heap order, then repeat for its children.

```
static boolean isHeapSlow(TreeNode root) {
    if(root==null)return true;
    return completeSlow(root)&&orderSlow(root)&&isHeapSlow(root.left)&&isHeapSlow(root.right);
}
static boolean orderSlow(TreeNode n){if(n==null)return true;if(n.left!=null&&n.val<n.left.val)return false;if(n.right!=null&&n.val<n.right.val)return false;return orderSlow(n.left)&&orderSlow(n.right);}
static boolean completeSlow(TreeNode root){Queue<TreeNode>q=new LinkedList<>();q.add(root);boolean gap=false;while(!q.isEmpty()){TreeNode n=q.remove();if(n==null){gap=true;continue;}if(gap)return false;q.add(n.left);q.add(n.right);}return true;}
```

2. Better: recursive completeness and heap summaries

$O(n)$ time, $O(h)$ stack

Return subtree height, whether it is full or complete, and heap order; combine the shape cases upward.

```
static class HeapInfo {int height;boolean full,complete,heap;HeapInfo(int h,boolean f,boolean c,boolean m){height=h;full=f;complete=c;heap=m;}}
static HeapInfo heapInfo(TreeNode n) {
    if(n==null)return new HeapInfo(0,true,true,true);HeapInfo l=heapInfo(n.left),r=heapInfo(n.right);
    boolean order=(n.left==null||n.val>=n.left.val)&&(n.right==null||n.val>=n.right.val)&&l.heap&&r.heap;
    int h=1+Math.max(l.height,r.height);boolean full=l.full&&r.full&&l.height==r.height;
    boolean complete=full||(l.full&&r.complete&&l.height==r.height)||(!l.complete&&r.full&&l.height==r.height+1);
    return new HeapInfo(h,full,complete,order);
}
```

3. Best robust check: level-order completeness scan

$O(n)$ time, $O(\text{width})$ queue

After the first missing child position, no later node may appear. Check the max-heap condition at the same time.

```
static boolean isMaxHeap(TreeNode root) {
    if(root==null)return true;Queue<TreeNode>q=new ArrayDeque<>();q.add(root);boolean gap=false;
    while(!q.isEmpty()){
        TreeNode n=q.remove();
        if(n.left!=null){if(gap||n.val<n.left.val)return false;q.add(n.left);}else gap=true;
        if(n.right!=null){if(gap||n.val<n.right.val)return false;q.add(n.right);}else gap=true;
    }
    return true;
}
```

Tips and edge cases

A binary tree can satisfy parent ordering but fail completeness. Check duplicates as valid when parent equals child. The recursive completeness summary above uses a compact shape condition; level-order is easiest to audit.

LeetCode matches or related practice: [Check Completeness Of A Binary Tree](#)

36. Find a cycle in an undirected graph

Problem: Given an undirected graph with vertices $0..V-1$ and an edge list, return whether any cycle exists. Self-loops and parallel edges count as cycles under the usual multigraph interpretation.

Pattern: Cycle detection; in undirected DFS, ignore only the edge back to the parent. For edge lists, disjoint set detects whether an edge joins already-connected vertices.

1. Brute force: remove each edge and test connectivity

$O(VE^2)$ time for this edge-list scan

An edge lies on a cycle exactly when its endpoints remain connected after that edge is removed.

```
static boolean cycleByRemoval(int n,int[][] edges) {
    for(int skip=0;skip<edges.length;skip++){
        int[] seen=new int[n];ArrayDeque<Integer>q=new ArrayDeque<>();int
        s=edges[skip][0],t=edges[skip][1];q.add(s);seen[s]=1;
        while(!q.isEmpty()){int u=q.remove();for(int e=0;e<edges.length;e++){if(e!=skip){int
            v=-1;if(edges[e][0]==u)v=edges[e][1];else
            if(edges[e][1]==u)v=edges[e][0];if(v>=0&&seen[v]==0){seen[v]=1;q.add(v);}}
        }
        if(seen[t]==1)return true;
    }
    return false;
}
```

2. Better: DFS with parent edge

$O(V+E)$ time, $O(V+E)$ space

Build adjacency lists with edge IDs so a parallel edge to the parent is not accidentally ignored.

```
static boolean hasCycleDfs(int n,int[][] edges) {
    List<int[]> g=new List[n];for(int i=0;i<n;i++){g[i]=new ArrayList<>();
    for(int id=0;id<edges.length;id++){int u=edges[id][0],v=edges[id][1];g[u].add(new
        int[] {v,id});g[v].add(new int[] {u,id});}
    boolean[] seen=new boolean[n];for(int i=0;i<n;i++){if(!seen[i]&&dfsCycle(i,-1,g,seen))return
        true;return false;
    }
    static boolean dfsCycle(int u,int
        pe,List<int[]> g,boolean[] seen){seen[u]=true;for(int[] e:g[u]){if(e[1]==pe)continue;if(seen[e[0]]||dfsCycle(e[0],e[1],g,seen))return true;}return
        false;}
}
```

3. Best for an edge list: Disjoint Set Union

$O(E \alpha(V))$ time, $O(V)$ space

Every edge should connect two different components. If $\text{find}(u) == \text{find}(v)$, adding it closes a cycle.

```
static boolean hasCycleDsu(int n,int[][] edges) {
    int[] p=new int[n],rank=new int[n];for(int i=0;i<n;i++)p[i]=i;
    for(int[] e:edges){int a=findRoot(p,e[0]),b=findRoot(p,e[1]);if(a==b)return true;
        if(rank[a]<rank[b])p[a]=b;else if(rank[a]>rank[b])p[b]=a;else{p[b]=a;rank[a]++;}
    }
    return false;
    static int findRoot(int[] p,int x){return p[x]==x?x:(p[x]=findRoot(p,p[x]));}
}
```

Tips and edge cases

For a simple graph, duplicate edges may be invalid input; for a multigraph, edge IDs matter in DFS. DSU is concise for an edge list; DFS is more useful when the graph is already stored as adjacency lists.

LeetCode matches or related practice: [Graph Valid Tree](#)

37. Find the minimum number of edges in a graph path

Problem: Given an unweighted graph and vertices source and target, return the minimum number of edges on a path between them, or -1 if no path exists.

Pattern: Breadth-first search; first discovery of a vertex is at minimum edge distance in an unweighted graph.

1. Brute force: enumerate simple paths

Exponential time

Explore every simple path from source and retain the one with fewest edges.

```
static int minPathBrute(List<Integer>[] g,int u,int target,boolean[] seen) {
    if(u==target)return 0;seen[u]=true;int best=Integer.MAX_VALUE;
    for(int v:g[u])if(!seen[v]){int
        d=minPathBrute(g,v,target,seen);if(d!=Integer.MAX_VALUE)best=Math.min(best,d+1);}
    seen[u]=false;return best;
}
```

2. Better: BFS over an adjacency matrix

$O(V^2)$ time, $O(V)$ queue/distances

BFS is already the right algorithm; scanning a full matrix for each removed vertex costs V^2 .

```
static int minEdgesMatrix(boolean[][] edge,int s,int t) {
    int n=edge.length;int[] d=new int[n];Arrays.fill(d,-1);Queue<Integer>q=new
    ArrayDeque<>();d[s]=0;q.add(s);
    while(!q.isEmpty()){int u=q.remove();if(u==t)return d[u];for(int
        v=0;v<n;v++)if(edge[u][v]&&d[v]<0){d[v]=d[u]+1;q.add(v);}}
    return -1;
}
```

3. Best: BFS with adjacency lists

$O(V+E)$ time, $O(V)$ auxiliary space

Visit each vertex and edge at most once; store distance or parent when a vertex is first discovered.

```
static int minEdges(List<Integer>[] g,int s,int t) {
    int[] d=new int[g.length];Arrays.fill(d,-1);Queue<Integer>q=new
    ArrayDeque<>();d[s]=0;q.add(s);
    while(!q.isEmpty()){int u=q.remove();if(u==t)return d[u];for(int
        v:g[u])if(d[v]<0){d[v]=d[u]+1;q.add(v);}}
    return -1;
}
```

Tips and edge cases

BFS assumes every edge has equal cost. If edge weights differ, use Dijkstra for nonnegative weights. If the problem asks for the actual path, store a parent for every first-discovered vertex.

LeetCode matches or related practice: [Shortest Path In Binary Matrix](#)

38. Find a path in a directed graph

Problem: Given a directed graph, source s , and target t , return whether any directed path from s to t exists. The optimized version below returns a shortest path in edge count.

Pattern: Reachability search; follow outgoing edges while marking discovered vertices so cycles cannot cause infinite recursion.

1. Brute force: enumerate simple directed paths

Exponential time

Backtrack through every not-yet-visited outgoing neighbor until the target is found.

```
static boolean pathBrute(List<Integer>[] g,int u,int t,boolean[] onPath) {
    if(u==t)return true;onPath[u]=true;
    for(int v:g[u])if(!onPath[v]&&pathBrute(g,v,t,onPath)){onPath[u]=false;return true;}
    onPath[u]=false;return false;
}
```

2. Better: depth-first reachability

$O(V+E)$ time, $O(V)$ space

Use a global visited set; once a node has been fully explored, revisiting it cannot reveal a new path.

```
static boolean hasDirectedPathDfs(List<Integer>[] g,int s,int t) {
    boolean[] seen=new boolean[g.length];return reachDfs(g,s,t,seen);
}
static boolean reachDfs(List<Integer>[] g,int u,int t,boolean[] seen){if(u==t)return true;seen[u]=true;for(int v:g[u])if(!seen[v]&&reachDfs(g,v,t,seen))return true;return false;}
}
```

3. Best when a shortest route is useful: BFS with parents

$O(V+E)$ time, $O(V)$ space

BFS finds the shortest path in an unweighted directed graph and parent links reconstruct one route.

```
static List<Integer> directedPath(List<Integer>[] g,int s,int t) {
    int[] par=new int[g.length];Arrays.fill(par,-1);Queue<Integer>q=new
    ArrayDeque<>();par[s]=s;q.add(s);
    while(!q.isEmpty()&&par[t]<0){int u=q.remove();for(int
    v:g[u])if(par[v]<0){par[v]=u;q.add(v);}}
    if(par[t]<0)return Collections.emptyList();List<Integer>p=new ArrayList<>();for(int
    v=t;v!=s;v=par[v])p.add(v);p.add(s);Collections.reverse(p);return p;
}
```

Tips and edge cases

A directed edge $u \rightarrow v$ does not imply $v \rightarrow u$. If only existence is required, DFS and BFS have the same asymptotic time; BFS adds shortest-path and reconstruction behavior.

LeetCode matches or related practice: [Find If Path Exists In Graph](#)

39. Count the number of islands

Problem: Given a grid of 1s (land) and 0s (water), count connected land components using horizontal and vertical adjacency.

Pattern: Flood fill / connected components; when a new unvisited land cell is found, mark every reachable land cell as part of that island.

1. Brute force: flood from every land cell and deduplicate component IDs

$O((mn)^2)$ time, $O(mn)$ per flood

For each land cell, search its full connected component, record its smallest linear index, and count distinct representatives.

```
static int islandsBrute(char[][] g) {
    int m=g.length;if(m==0)return 0;int n=g[0].length;Set<Integer> reps=new HashSet<>();
    for(int r=0;r<m;r++)for(int c=0;c<n;c++)if(g[r][c]=='1'){
        boolean[][] seen=new boolean[m][n];ArrayDeque<int[]>q=new ArrayDeque<>();q.add(new
            int[]{r,c});seen[r][c]=true;int min=r*n+c;
        while(!q.isEmpty()){int[]x=q.remove();min=Math.min(min,x[0]*n+x[1]);for(int[]d:new
            int[][]{{1,0},{-1,0},{0,1},{0,-1}}){int
                a=x[0]+d[0],b=x[1]+d[1];if(a>=0&&b<=m-1&&a<m&&b<n&&g[a][b]=='1'&&!seen[a][b]){seen[a][b]=true;q.add(new int[]{a,b});}}
            reps.add(min);
        }
    }
    return reps.size();
}
```

2. Better: recursive DFS flood fill

$O(mn)$ time, $O(mn)$ worst-case recursion

Start a flood fill at each unseen island and mark cells in place to avoid processing them again.

```
static int islandsDfs(char[][] g) {
    int count=0;for(int r=0;r<g.length;r++)for(int
        c=0;c<g[0].length;c++)if(g[r][c]=='1'){count++;sink(g,r,c);}return count;
    }
    static void sink(char[][]g,int r,int
        c){if(r<0||c<0||r==g.length||c==g[0].length||g[r][c]!='1')return;g[r][c]='0';sink(g,r+1,c);sink(g,r-1,c);sink(g,r,c+1);sink(g,r,c-1);}
}
```

3. Best robust traversal: iterative BFS

$O(mn)$ time, $O(mn)$ worst-case queue

Use a queue instead of recursion to avoid stack overflow on a large connected grid.

```
static int numIslands(char[][] g) {
    int m=g.length;if(m==0)return 0;int n=g[0].length,count=0;int[]dr={1,-1,0,0},dc={0,0,1,-1};
    for(int r=0;r<m;r++)for(int c=0;c<n;c++)if(g[r][c]=='1'){
        count++;g[r][c]='0';Queue<int[]>q=new ArrayDeque<>();q.add(new int[]{r,c});
        while(!q.isEmpty()){int[]x=q.remove();for(int i=0;i<4;i++){int
            a=x[0]+dr[i],b=x[1]+dc[i];if(a>=0&&b<=m-1&&a<m&&b<n&&g[a][b]=='1'&&g[a][b]!='0'){g[a][b]='0';q.add(new int[]{a,b});}}
        }
    }
    return count;
}
```

Tips and edge cases

These methods mutate the grid by sinking visited land. Clone rows first if the original must be preserved. Use eight directions only if diagonal contact counts as connected.

LeetCode matches or related practice: [Number Of Islands](#)

40. Shortest paths using Dijkstra's algorithm

Problem: Given a directed or undirected weighted graph with nonnegative edge weights and source s , return shortest distances from s to all vertices. Unreachable vertices remain infinity.

Pattern: Greedy shortest path; repeatedly finalize the closest unsettled vertex and relax its outgoing edges.

1. Brute force: enumerate simple paths from source

Exponential time

For each target, enumerate every simple path and take the smallest total weight.

```
static long shortestBrute(List<int[]> g,int s,int t,boolean[] seen) {
    if(s==t)return 0;seen[s]=true;long best=Long.MAX_VALUE/4;
    for(int[] e:g[s])if(!seen[e[0]]){long
        d=shortestBrute(g,e[0],t,seen);if(d<Long.MAX_VALUE/4)best=Math.min(best,d+e[1]);}
    seen[s]=false;return best;
}
```

2. Better: array-based Dijkstra

$O(V^2+E)$ time, $O(V)$ auxiliary space

At each iteration, scan all unvisited vertices for the smallest tentative distance.

```
static long[] dijkstraArray(List<int[]> g,int s) {
    int n=g.length;long inf=Long.MAX_VALUE/4;long[] d=new long[n];boolean[] done=new
        boolean[n];Arrays.fill(d,inf);d[s]=0;
    for(int z=0;z<n;z++){int u=-1;for(int i=0;i<n;i++){if(!done[i]&&(u<0||d[i]<d[u]))u=i;
        if(u<0||d[u]==inf)break;done[u]=true;for(int[] e:g[u])if(d[u]+e[1]<d[e[0]])d[e[0]]=d[u]+e[1];}
    return d;
}
```

3. Best for sparse graphs: priority queue

$O((V+E) \log V)$ time, $O(V+E)$ space

Push improved distance labels into a min-heap. Ignore stale entries when removed.

```
static long[] dijkstra(List<int[]> g,int s) {
    int n=g.length;long inf=Long.MAX_VALUE/4;long[] d=new long[n];Arrays.fill(d,inf);d[s]=0;
    PriorityQueue<long[]> pq=new PriorityQueue<>(Comparator.comparingLong(x->x[1]));pq.add(new
        long[]{s,0});
    while(!pq.isEmpty()){long[] x=pq.remove();int u=(int)x[0];long du=x[1];if(du!=d[u])continue;
        for(int[] e:g[u]){int v=e[0];long nd=du+e[1];if(nd<d[v]){d[v]=nd;pq.add(new
            long[]{v,nd});}}
    }
    return d;
}
```

Tips and edge cases

Dijkstra requires nonnegative edge weights. For a single target, stop only when that target is removed from the priority queue with its current best distance. Use Bellman–Ford if negative weights are allowed.

LeetCode matches or related practice: [Network Delay Time](#)